

Java Collections — Cheat Sheet

Thread Safety · Vector vs ArrayList · ArrayDeque vs Stack

1. Thread-Safe Collections

Thread safety means that a collection can be safely accessed by multiple threads at the same time without causing data corruption. In Java, the following collections are thread-safe by default:

Collection	Thread-Safe?	Reason
Vector	✓ Yes	All methods are synchronized
Stack	✓ Yes	Extends Vector — inherits synchronization
Hashtable	✓ Yes	All methods are synchronized
ArrayList	✗ No	Not synchronized — faster but not safe
ArrayDeque	✗ No	Not synchronized — use with single thread
LinkedList	✗ No	Not synchronized
CopyOnWriteArrayList	✓ Yes	Creates a new copy of the array on each write

Note: For modern multi-threaded code, prefer CopyOnWriteArrayList or Collections.synchronizedList() over Vector.

2. Vector vs ArrayList

Both Vector and ArrayList implement the List interface and store elements in a dynamic array. The key difference is that Vector is synchronized (thread-safe) while ArrayList is not, which makes ArrayList faster in single-threaded environments.

Feature	Vector	ArrayList
Thread-Safe	✓ Yes	✗ No
Speed	Slower (synchronized overhead)	Faster
Capacity Growth	Doubles in size (× 2)	Grows by 50% (× 1.5)
Introduced	Java 1.0 — Legacy	Java 1.2 — Modern
Null values	✓ Yes	✓ Yes
Best used when	Legacy code / multi-thread needed	General purpose, single thread

Recommendation: Use ArrayList in most cases. If thread safety is needed, use Collections.synchronizedList(new ArrayList<>()) instead of Vector.

3. ArrayDeque vs Stack

Both ArrayDeque and Stack can be used as a LIFO (Last-In, First-Out) data structure. Stack is a legacy class that extends Vector, which means it carries unnecessary synchronization overhead. ArrayDeque is the modern, faster alternative recommended by Java's own documentation.

Feature	ArrayDeque	Stack
Thread-Safe	✗ No	✓ Yes
Speed	Faster	Slower (synchronized)
Null values	✗ No	✓ Yes
Usage Pattern	LIFO and FIFO (flexible)	LIFO only
Extends	AbstractCollection	Vector — Legacy
Introduced	Java 6 — Modern	Java 1.0 — Legacy
Best used when	Any stack or queue scenario	Legacy code only

Recommendation: Always prefer ArrayDeque over Stack. Java's documentation explicitly recommends ArrayDeque as the better stack implementation.

4. Quick Reference — What to Use?

Situation	Best Choice
Simple list, single thread	ArrayList
Simple list, multiple threads	CopyOnWriteArrayList
Stack (LIFO), single thread	ArrayDeque
Queue (FIFO), single thread	ArrayDeque
Queue (FIFO), multiple threads	LinkedBlockingQueue
Key-value map, single thread	HashMap
Key-value map, multiple threads	ConcurrentHashMap